



UNIVERSIDADE FEDERAL DO CEARÁ
Campus Quixadá
Curso: Redes de Computadores
Disciplina: Redes

Aula 5 – Camada de Aplicação (HTTP)

Prof. Rafael Braga

Capítulo 2: Roteiro

- 2.1 Princípios de aplicações de rede
- 2.2 A Web e o HTTP
- 2.3 Transferência de arquivo: FTP
- 2.4 Correio Eletrônico na Internet
- 2.5 DNS: o serviço de diretório da Internet
- 2.6 Aplicações P2P
- 2.7 Programação de *sockets* com TCP
- 2.8 Programação de *sockets* com UDP
- 2.9 Construindo um servidor Web simples

A Web e o HTTP

Primeiro algum jargão

- **Páginas Web** consistem de **objetos**
- Objeto pode ser um arquivo HTML, uma imagem JPEG, um applet Java, um arquivo de áudio,...
- **Páginas Web** consistem de um **arquivo base HTML** que inclui vários objetos referenciados
- Cada objeto é endereçável por uma **URL**
- Exemplo de URL:

`www.someschool.edu/someDept/pic.gif`

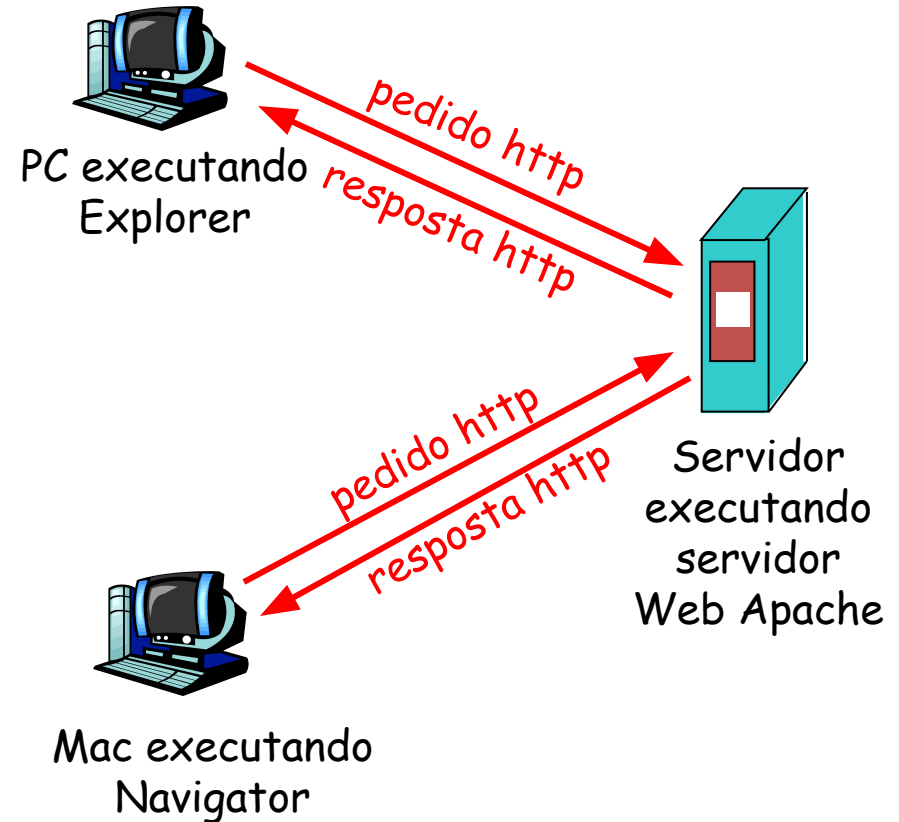
nome do hospedeiro

nome do caminho

Protocolo HTTP

HTTP: *hypertext transfer protocol*

- protocolo da camada de aplicação da Web
- modelo cliente/servidor
 - *cliente*: browser que pede, recebe, “visualiza” objetos Web
 - *servidor*: servidor Web envia objetos em resposta a pedidos



Mais sobre o protocolo HTTP

Usa serviço de transporte TCP:

- ❑ cliente inicia conexão TCP (cria *socket*) ao servidor, porta 80
- ❑ servidor aceita conexão TCP do cliente
- ❑ mensagens HTTP (mensagens do protocolo da camada de apl) trocadas entre *browser* (cliente HTTP) e servidor Web (servidor HTTP)
- ❑ encerra conexão TCP

HTTP é “sem estado”

- ❑ servidor não mantém informação sobre pedidos anteriores do cliente

Nota
Protocolos que mantêm “estado” são complexos!

- ❑ história passada (estado) tem que ser guardada
- ❑ Caso caia servidor/cliente, suas visões do “estado” podem ser inconsistentes, devem ser reconciliadas

Conexões HTTP

HTTP não persistente

- No máximo um objeto é enviado numa conexão TCP

HTTP persistente

- Múltiplos objetos podem ser enviados sobre uma única conexão TCP entre cliente e servidor

Exemplo de HTTP não persistente

Suponha que usuário digite a URL a seguir que contém texto e referências a 10 imagens jpeg: www.algumaUniv.br/algumDepartamento/inicial.index

tempo

1a. Cliente http inicia conexão TCP a servidor http (processo) a www.algumaUniv.br. Porta 80 é padrão para servidor http.

1b. servidor http no hospedeiro www.algumaUniv.br espera por conexão TCP na porta 80. "aceita" conexão, avisando ao cliente

2. cliente http envia *mensagem de pedido* de http (contendo URL) através do socket da conexão TCP. A mensagem indica que o cliente deseja receber o objeto [algumDepartamento/inicial.index](http://www.algumaUniv.br/algumDepartamento/inicial.index)

3. servidor http recebe mensagem de pedido, formula *mensagem de resposta* contendo objeto solicitado e envia a mensagem via socket

Exemplo de HTTP não persistente (cont.)

5. cliente http recebe mensagem de resposta contendo arquivo html, visualiza html. Analisando arquivo html, encontra 10 objetos jpeg referenciados

4. servidor http encerra conexão TCP .



6. Passos 1 a 5 repetidos para cada um dos 10 objetos jpeg

tempo

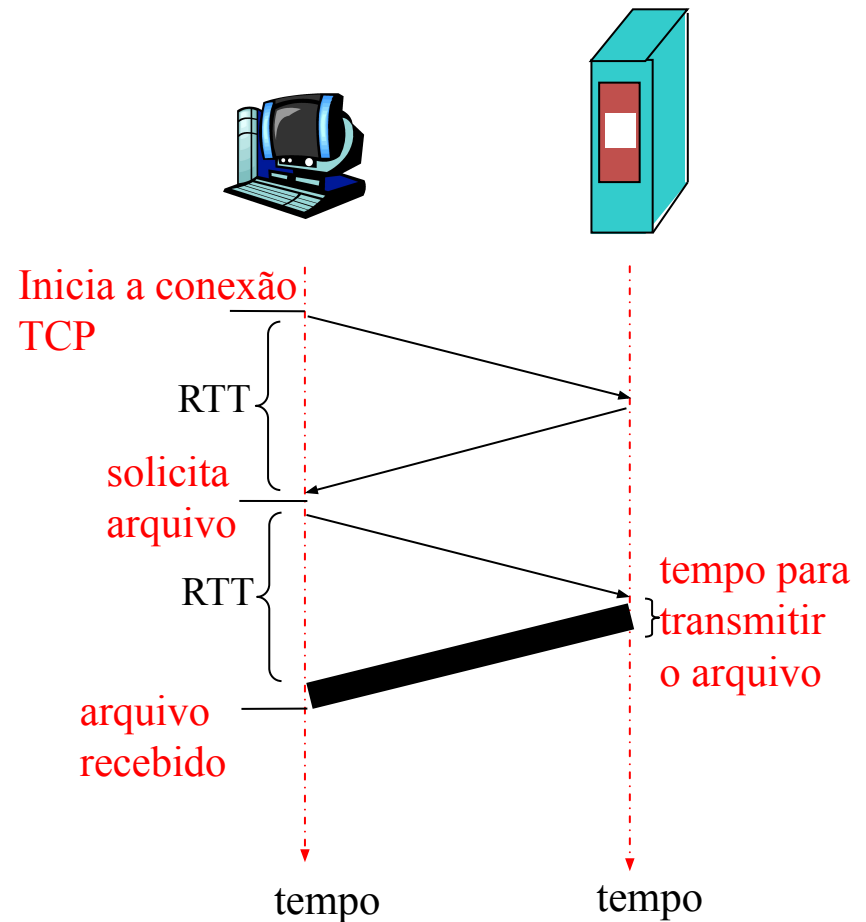
Modelagem do tempo de resposta

Definição de RTT (*Round Trip Time*):

intervalo de tempo entre a ida e a volta de um pequeno pacote entre um cliente e um servidor

Tempo de resposta:

- um RTT para iniciar a conexão TCP
 - um RTT para o pedido HTTP e o retorno dos primeiros bytes da resposta HTTP
 - tempo de transmissão do arquivo
- total = $2RTT + \text{tempo de transmissão}$**



HTTP persistente

Problemas com o HTTP não persistente:

- requer 2 RTTs para cada objeto
- SO aloca recursos do hospedeiro para cada conexão TCP
- os *browsers* freqüentemente abrem conexões TCP paralelas para recuperar os objetos referenciados

HTTP persistente

- o servidor deixa a conexão aberta após enviar a resposta
- mensagens HTTP seguintes entre o mesmo cliente/servidor são enviadas nesta conexão
- o cliente envia os pedidos logo que encontra um objeto referenciado
- pode ser necessário apenas um RTT para todos os objetos referenciados

Formato da mensagem HTTP: requisição

- Dois tipos de mensagem HTTP: *requisição, resposta*
- **mensagem de requisição HTTP:**
 - ASCII (formato legível por pessoas)

linha da requisição
(comandos GET,
POST, HEAD)

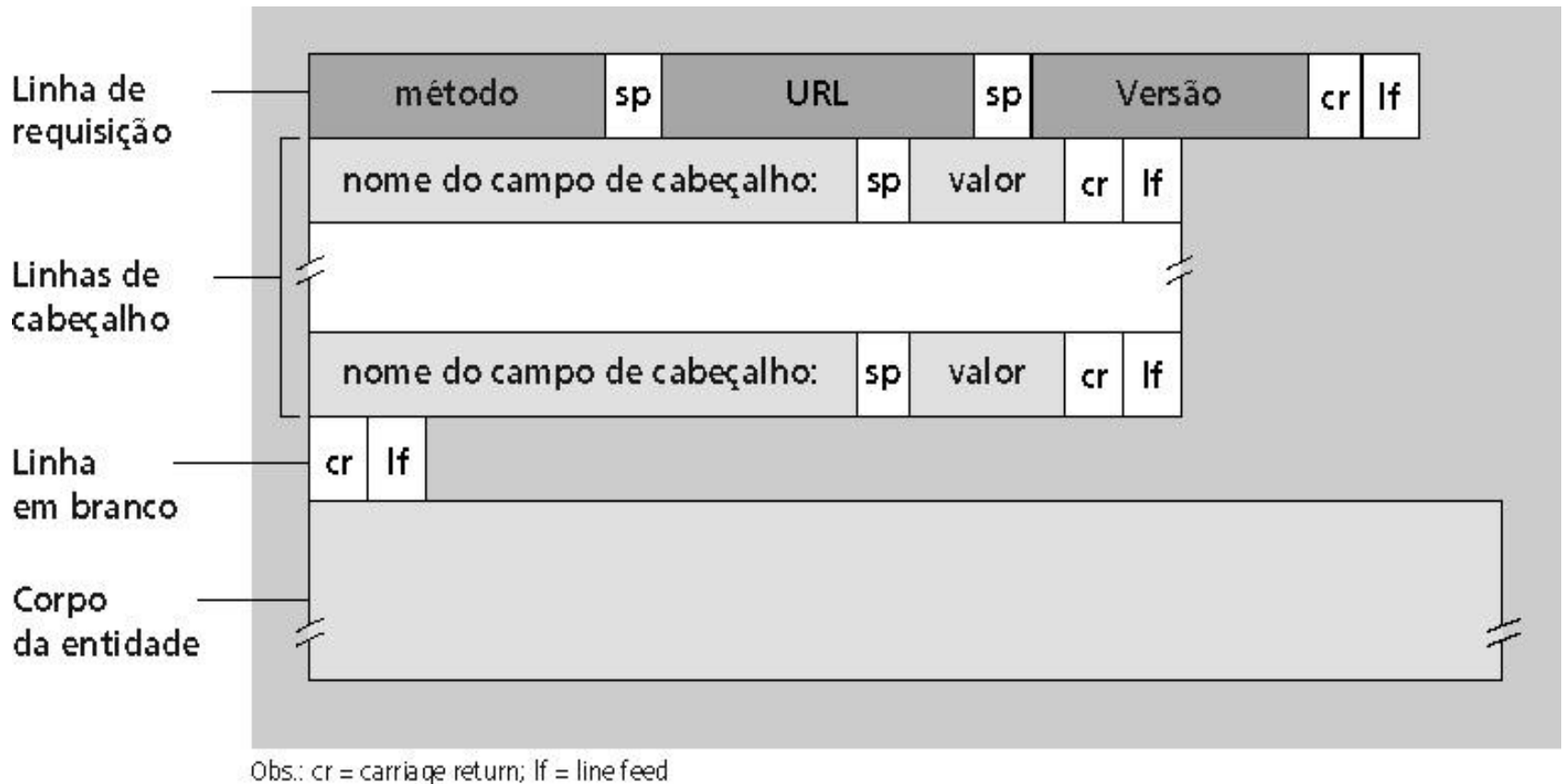
linhas de
cabeçalho

```
GET /somedir/page.html HTTP/1.0
Host: www.someschool.edu
User-agent: Mozilla/4.0
Connection: close
Accept-language: fr
```

Carriage return,
line feed o indicam
fim de mensagem

→ (carriage return (CR), line feed(LF) adicionais)

Mensagem de requisição HTTP: formato geral



Enviando conteúdo de formulário

Método POST :

- Páginas Web frequentemente contêm formulário de entrada
- Conteúdo é enviado para o servidor no corpo da mensagem

Método URL:

- Usa o método GET
- Conteúdo é enviado para o servidor no campo URL:

`www.somesite.com/animalsearch?key=monkeys&max=10`

Tipos de métodos

HTTP/1.0

- GET
- POST
- HEAD
 - Pede para o servidor não enviar o objeto requerido junto com a resposta

HTTP/1.1

- GET, POST, HEAD
- PUT
 - *Upload* de arquivo contido no corpo da mensagem para o caminho especificado no campo URL
- DELETE
 - Exclui arquivo especificado no campo URL

Formato de mensagem HTTP: resposta

linha de status
(protocolo,
código de status,
frase de status)

linhas de
cabeçalho

dados, p.ex.,
arquivo html
solicitado

HTTP/1.1 200 OK

Connection close

Date: Thu, 06 Aug 1998 12:00:15 GMT

Server: Apache/1.3.0 (Unix)

Last-Modified: Mon, 22 Jun 1998

Content-Length: 6821

Content-Type: text/html

dados dados dados dados ...

Códigos de status da resposta HTTP

Na primeira linha da mensagem de resposta servidor->cliente.
Alguns códigos típicos:

200 OK

- sucesso, objeto pedido segue mais adiante nesta mensagem

301 Moved Permanently

- objeto pedido mudou de lugar, nova localização especificado mais adiante nesta mensagem (Location:)

400 Bad Request

- mensagem de pedido não entendida pelo servidor

404 Not Found

- documento pedido não se encontra neste servidor

505 HTTP Version Not Supported

- versão de http do pedido não usada por este servidor

Experimente você com HTTP (do lado cliente)

1. Use cliente telnet para seu servidor WWW favorito:

```
telnet www.ufc.br 80
```

Abre conexão TCP para a porta 80 (porta padrão do servidor http) a www.ic.uff.br. Qualquer coisa digitada é enviada para a porta 80 do www.ic.uff.br

2. Digite um pedido GET HTTP:

```
GET /a-universidade HTTP/1.0
```

Digitando isto (deve teclar ENTER duas vezes), está enviando este pedido GET mínimo (porém completo) ao servidor http

3. Examine a mensagem de resposta enviada pelo servidor HTTP !...

Experimente você com HTTPS (do lado cliente)

1. Use cliente telnet para seu servidor WWW favorito:

```
openssl s_client -connect www.quixada.ufc.br:443
```

Abre conexão TCP para a porta 443
(porta padrão do servidor https) a www.quixada.ufc.br.
O certificado é exibido
Qualquer coisa digitada é enviada para a
porta 443 do www.quixada.ufc.br

2. Digite um pedido GET HTTP:

```
GET /docente/marcos-dantas-ortiz/ HTTP/1.0  
Host: www.quixada.ufc.br
```

Digitando isto (deve teclar
ENTER duas vezes), está enviando
este pedido GET mínimo (porém
completo) ao servidor http

3. Examine a mensagem de resposta enviada pelo
servidor HTTP !

Cookies: manutenção do “estado” da conexão

Muitos dos principais sítios
Web usam *cookies*

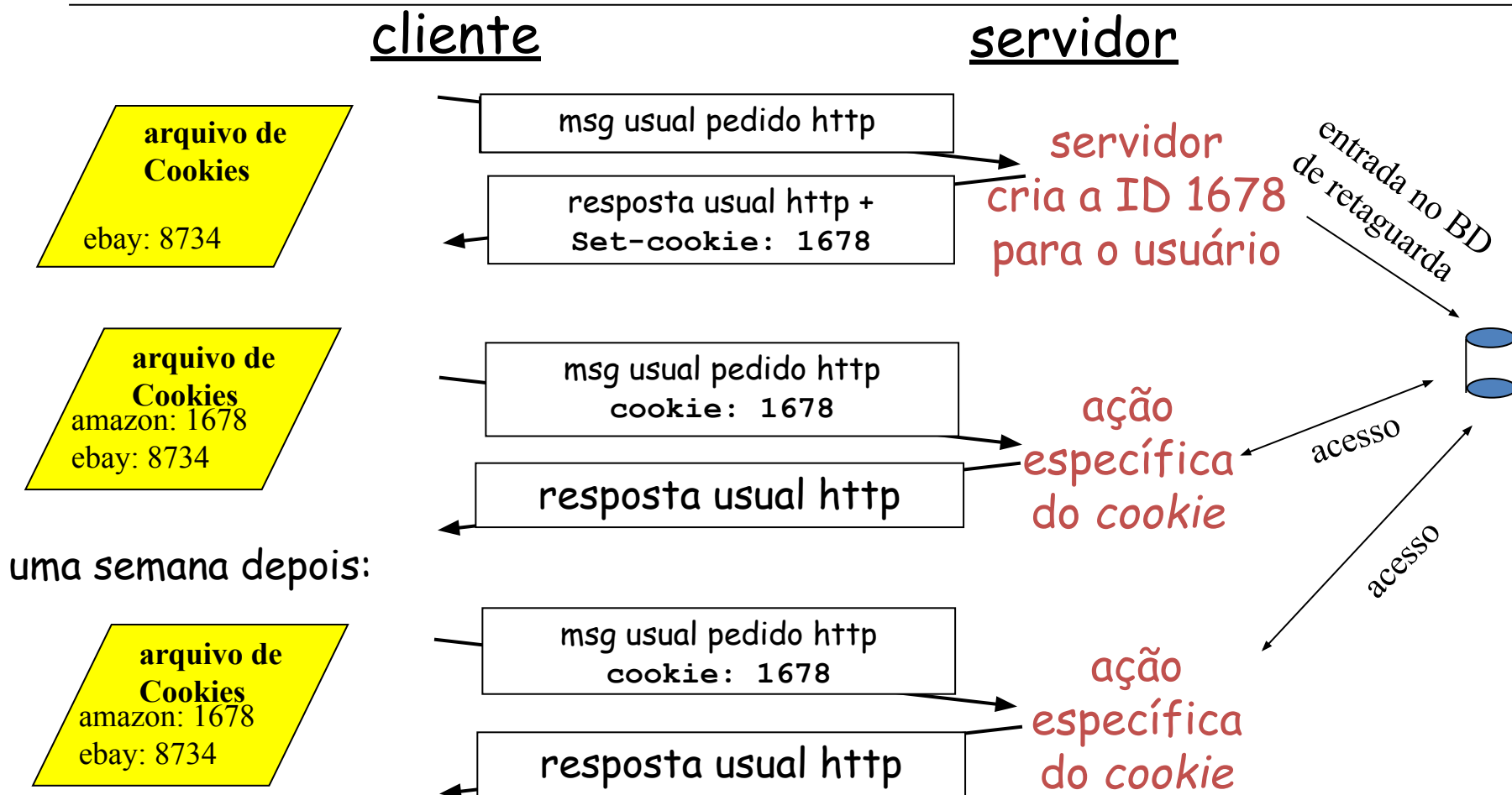
Quatro componentes:

- 1) linha de cabeçalho do *cookie* na mensagem de resposta HTTP
- 2) linha de cabeçalho do *cookie* na mensagem de pedido HTTP
- 3) arquivo do *cookie* mantido no host do usuário e gerenciado pelo browser do usuário
- 4) BD de retaguarda no sítio Web

Exemplo:

- Suzana acessa a Internet sempre do mesmo PC
- Ela visita um sítio específico de comércio eletrônico pela primeira vez
- Quando os pedidos iniciais HTTP chegam no sítio, o sítio cria
 - uma ID única
 - uma entrada para a ID no BD de retaguarda

Cookies: manutenção do “estado” (cont.)



Cookies (continuação)

nota

O que os cookies podem obter:

- ☐ autorização
- ☐ carrinhos de compra
- ☐ sugestões
- ☐ estado da sessão do usuário
(Webmail)

Cookies e privacidade:

- ☐ cookies permitem que os sítios aprendam muito sobre você
- ☐ você pode fornecer nome e e-mail para os sítios

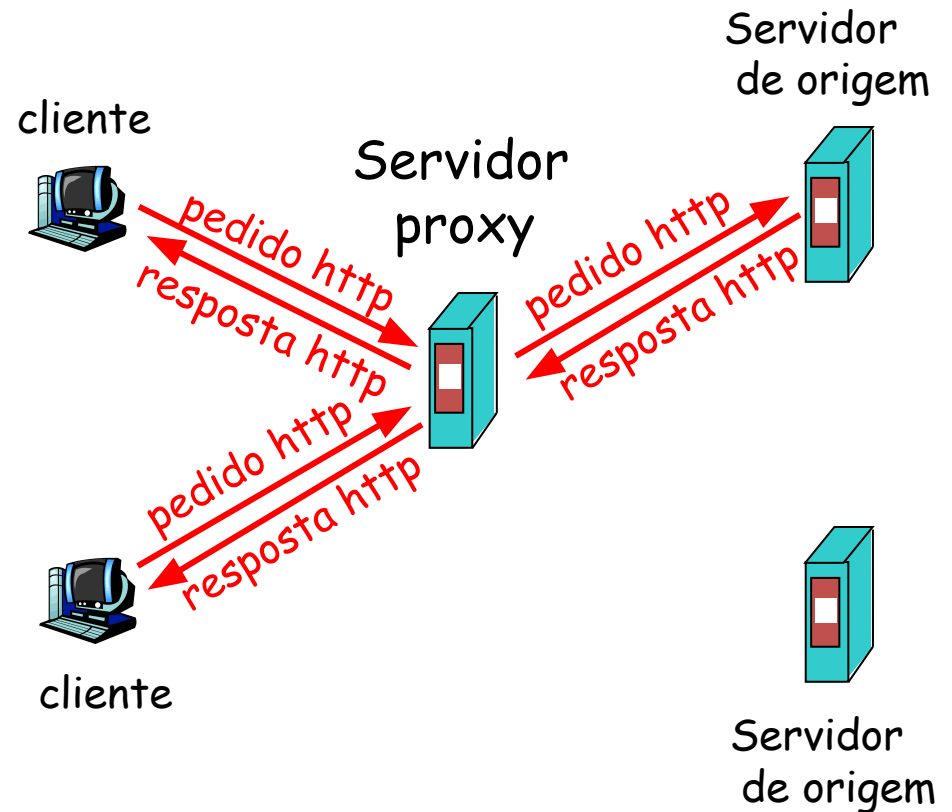
Como manter o "estado":

- ☐ Pontos finais do protocolo: mantêm o estado no transmissor/receptor para múltiplas transações
- ☐ *Cookies*: mensagens http transportam o estado

Cache Web (servidor proxy)

Meta: atender pedido do cliente sem envolver servidor de origem

- usuário configura browser: acessos Web via proxy
- cliente envia todos pedidos HTTP ao proxy
 - se objeto estiver no cache do proxy, este o devolve imediatamente na resposta HTTP
 - senão, solicita objeto do servidor de origem, depois devolve resposta HTTP ao cliente



Mais sobre Caches Web

- Cache atua tanto como cliente quanto como servidor
- Tipicamente o cache é instalado por um ISP (universidade, empresa, ISP residencial)

Para que fazer cache Web?

- Redução do tempo de resposta para os pedidos do cliente
- Redução do tráfego no canal de acesso de uma instituição
- A Internet cheia de caches permitem que provedores de conteúdo “pobres” efetivamente forneçam conteúdo (mas o compartilhamento de arquivos P2P também!)

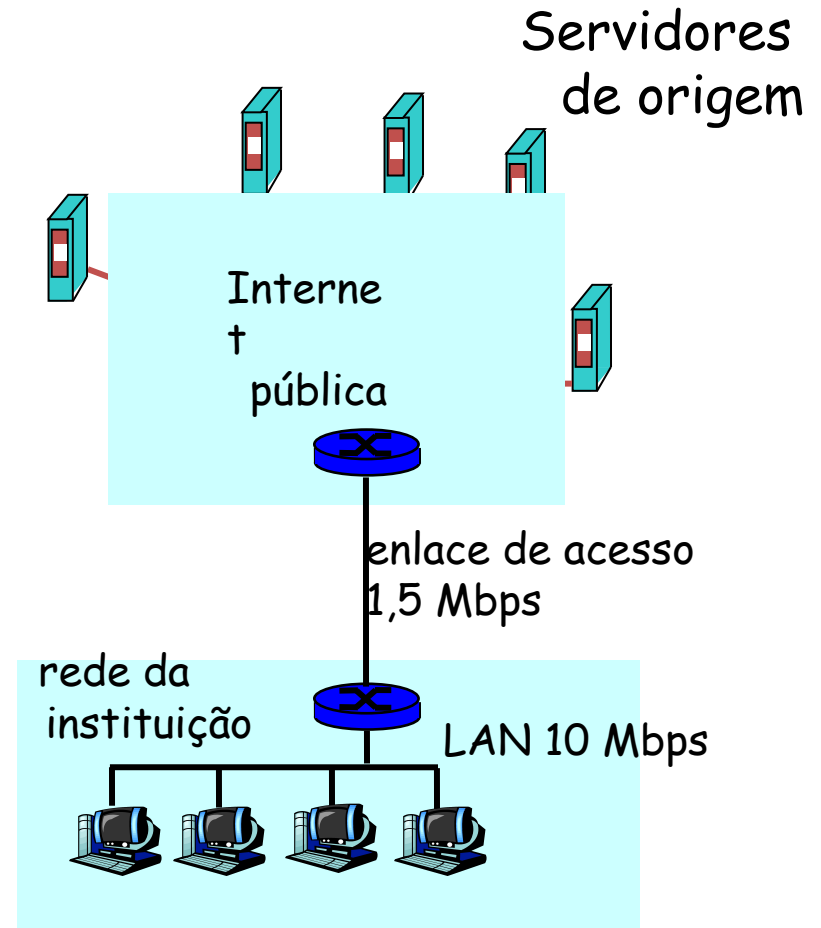
Exemplo de cache (1)

Hipóteses

- Tamanho médio de um objeto = 100.000 bits
- Taxa média de solicitações dos *browsers* de uma instituição para os servidores originais = 15/seg
- Atraso do roteador institucional para qualquer servidor origem e de volta ao roteador = 2seg

Consequências

- Utilização da LAN = 15%
- Utilização do canal de acesso = 100%
- Atraso total = atraso da Internet + atraso de acesso + atraso na LAN = 2 seg + minutos + milisegundos



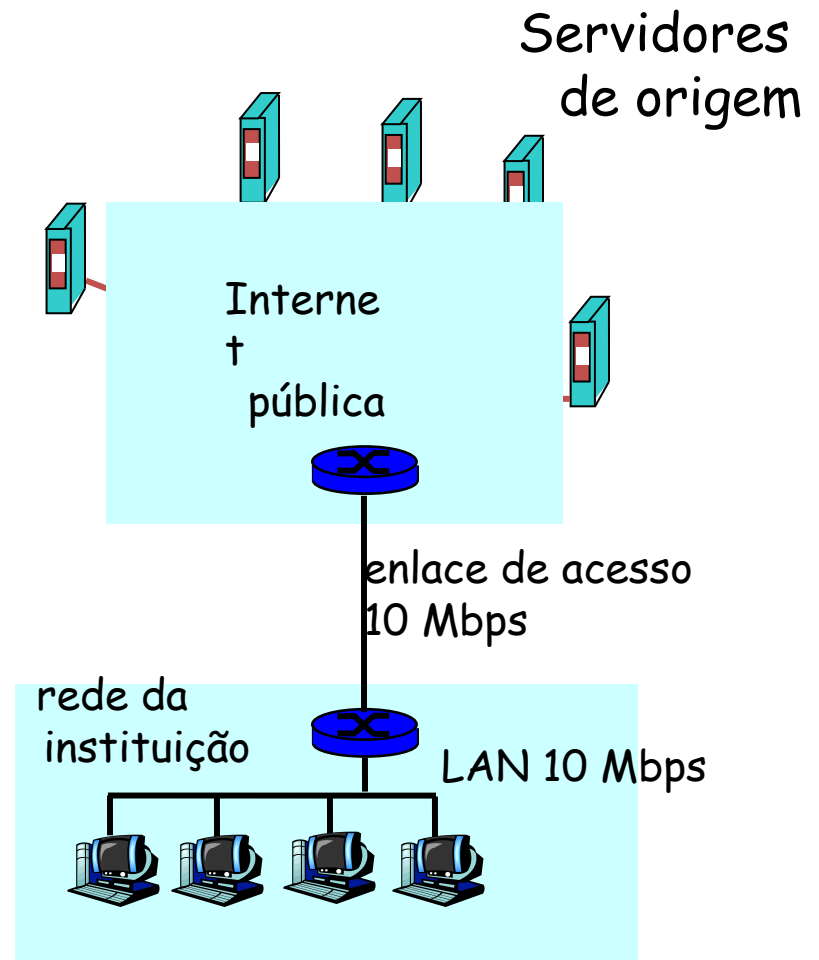
Exemplo de cache (2)

Solução em potencial

- Aumento da largura de banda do canal de acesso para, por exemplo, 10 Mbps

Consequências

- Utilização da LAN = 15%
- Utilização do canal de acesso = 15%
- Atraso total = atraso da Internet + atraso de acesso + atraso na LAN = 2 seg + msecs + msecs
- Frequentemente este é uma ampliação cara



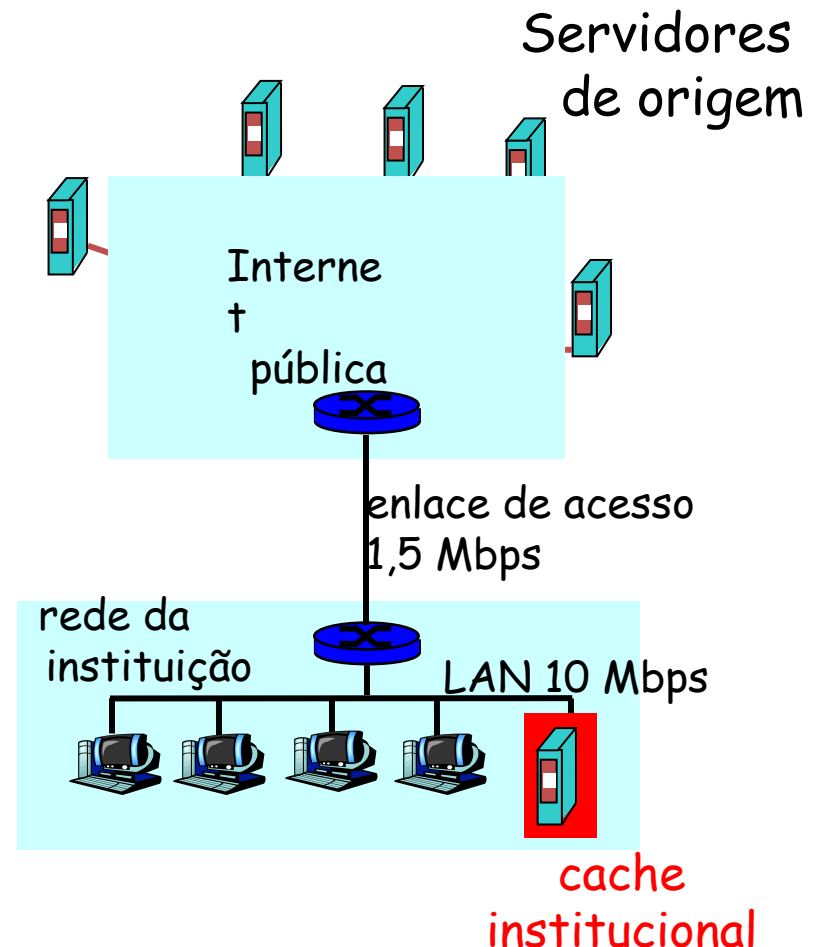
Exemplo de cache (3)

Instale uma cache

- Assuma que a taxa de acerto seja de 0,4

Conseqüências

- 40% dos pedidos serão atendidos quase que imediatamente
- 60% dos pedidos serão servidos pelos servidores de origem
- Utilização do canal de acesso é reduzido para 60%, resultando em atrasos desprezíveis (ex. 10 mseg)
- Atraso total = atraso da Internet + atraso de acesso + atraso na LAN = $0,6 \cdot 2 \text{ seg} + 0,6 \cdot 0,01 \text{ segs} + \text{msegs} < 1,3 \text{ segs}$



GET condicional

- **Meta:** não enviar objeto se cliente já tem (no cache) versão atual
- cache: especifica data da cópia no cache no pedido http

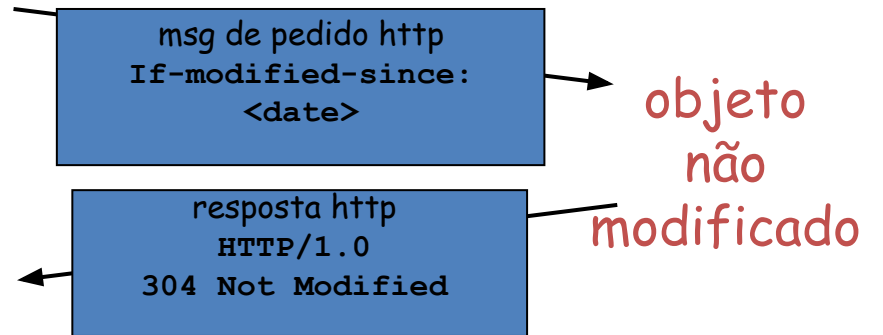
If-modified-since: <date>

- servidor: resposta não contém objeto se cópia no cache é atual:

HTTP/1.0 304 Not Modified

cache

servidor



.....

